



Northbridge Group — Enterprise Data Engineering & Financial Analytics Pipeline

Role: Senior BI Consultant / Analytics Engineer

Objective: Audit, transform, and model operational & financial datasets (2024–2025) to evaluate entity-level gross margins, identify revenue leakage, reconcile billing to the General Ledger, and produce a Star Schema for Power BI executive reporting.

1. Environment Setup & Data Ingestion Audit

What this section does: Connects to the SQLite database (`Northbridge_Assessment.db`) and executes a primary key / table grain audit across all 8 tables. This step verifies row counts, schema definitions, and primary key uniqueness before running any business logic.

```
In [1]: import sqlite3
import pandas as pd
import numpy as np

# Set display options for clean notebook output
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)
pd.set_option('display.float_format', lambda x: '%.2f' % x)

# Connect to database
conn = sqlite3.connect('Northbridge_Assessment.db')

def audit_raw_tables(connection):
    """
    Audits table row counts, primary key uniqueness, and null value counts.
    """
    tables = [
        'entities', 'clients', 'projects', 'staff',
        'timesheets', 'invoices', 'invoice_lines', 'gl_revenue_summary'
    ]
```

```
audit_list = []

for tbl in tables:
    df = pd.read_sql_query(f"SELECT * FROM {tbl}", connection)

    # Determine primary key column name
    pk_cols = [c for col in ['id', 'code'] for c in df.columns if col in c.lower()]
    pk = pk_cols[0] if pk_cols else df.columns[0]

    total_rows = len(df)
    unique_keys = df[pk].nunique()
    total_nulls = df.isnull().sum().sum()

    audit_list.append({
        'Table Name': tbl,
        'Total Rows': total_rows,
        'Primary Key': pk,
        'Unique PK Count': unique_keys,
        'PK Unique?': total_rows == unique_keys,
        'Total Null Cells': total_nulls
    })

return pd.DataFrame(audit_list)

# Run raw audit
raw_audit_df = audit_raw_tables(conn)
raw_audit_df
```

Out[1]:

	Table Name	Total Rows	Primary Key	Unique PK Count	PK Unique?	Total Null Cells
0	entities	3	entity_code	3	True	0
1	clients	180	client_id	180	True	0
2	projects	640	project_id	640	True	99
3	staff	210	staff_id	210	True	0
4	timesheets	35634	timesheet_id	35634	True	2984
5	invoices	2339	invoice_id	2339	True	0
6	invoice_lines	7010	line_id	7010	True	0
7	gl_revenue_summary	72	entity_code	3	False	0

2. Data Quality Hygiene & Transformation Engine

What this section does: Addresses three critical data quality anomalies identified during the initial audit:

1. **String Whitespace Removal:** Trims whitespace from `entity_code` in `projects`, `staff`, and `gl_revenue_summary` to ensure zero-loss relational joins.
2. **Service Line Unification:** Consolidates duplicate naming variants (`M&E` → `Mechanical & Electrical` ; `Consulting` → `Advisory`).
3. **Contractor Rate Imputation:** Contractor positions (`engagement_type` = 'Contract') contain `NULL` cost rates in `timesheets` . We calculate the permanent employee average cost rate per `grade` (Analyst, Consultant, Senior Consultant, Principal, Director) and apply it to contractor hours to prevent gross margin distortion.

```
In [2]: def load_and_transform_data(connection):
        """
        Loads raw tables into Pandas DataFrames, cleans strings, unifies service lines,
        and imputes contractor cost rates using permanent staff grade averages.
        """
        # 1. Extract raw tables
        entities = pd.read_sql_query("SELECT * FROM entities", connection)
```

```
clients = pd.read_sql_query("SELECT * FROM clients", connection)
projects = pd.read_sql_query("SELECT * FROM projects", connection)
staff = pd.read_sql_query("SELECT * FROM staff", connection)
timesheets = pd.read_sql_query("SELECT * FROM timesheets", connection)
invoices = pd.read_sql_query("SELECT * FROM invoices", connection)
invoice_lines = pd.read_sql_query("SELECT * FROM invoice_lines", connection)
gl_revenue = pd.read_sql_query("SELECT * FROM gl_revenue_summary", connection)

# 2. Trim string whitespace
entities['entity_code'] = entities['entity_code'].str.strip()
projects['entity_code'] = projects['entity_code'].str.strip()
staff['entity_code'] = staff['entity_code'].str.strip()
gl_revenue['entity_code'] = gl_revenue['entity_code'].str.strip()

# 3. Unify duplicate service lines
service_line_map = {
    'M&E': 'Mechanical & Electrical',
    'Consulting': 'Advisory'
}
projects['clean_service_line'] = projects['service_line'].replace(service_line_map)

# 4. Contractor Cost Rate Imputation Engine
ts_staff = timesheets.merge(staff[['staff_id', 'grade', 'engagement_type']], on='staff_id', how='left')

# Calculate average permanent staff cost rate per grade
perm_grade_averages = (
    ts_staff[ts_staff['engagement_type'] == 'Permanent']
    .groupby('grade')['cost_rate']
    .mean()
    .to_dict()
)

# Impute missing cost rates
ts_staff['imputed_cost_rate'] = ts_staff.apply(
    lambda r: r['cost_rate'] if pd.notnull(r['cost_rate']) else perm_grade_averages.get(r['grade'], 0.0),
    axis=1
)

# Calculate known vs imputed costs
ts_staff['known_cost_gbp'] = ts_staff['hours'] * ts_staff['cost_rate'].fillna(0.0)
ts_staff['imputed_cost_gbp'] = ts_staff['hours'] * ts_staff['imputed_cost_rate']
```

```
print("✅ Transformation Engine Execution Successful!")
print(f"Contractor Rows Imputed: {ts_staff['cost_rate'].isnull().sum():,d} timesheet entries")

return {
    'entities': entities,
    'clients': clients,
    'projects': projects,
    'staff': staff,
    'timesheets': ts_staff,
    'invoices': invoices,
    'invoice_lines': invoice_lines,
    'gl_revenue': gl_revenue
}

# Execute ETL Pipeline
data_dict = load_and_transform_data(conn)
```

✅ Transformation Engine Execution Successful!
Contractor Rows Imputed: 2,984 timesheet entries

3. Financial Analytics & Business Intelligence (Part A Deliverables)

What this section does: Executes core analytical calculations for Part A:

- **FY2025 Entity Performance:** Invoiced Revenue, Total Delivery Hours, Permanent vs. Contractor Hours, Direct Delivery Costs, and Gross Margin % (Known vs. Imputed)[cite: 3].
- **FY2025 Service Line Profitability:** Aggregates financial performance across cleaned service lines[cite: 3].
- **Year-over-Year Trend Analysis (FY2024 vs. FY2025):** Evaluates revenue growth against delivery cost expansion[cite: 3].
- **General Ledger Reconciliation:** Compares operational invoice revenue against `gl_revenue_summary` to quantify revenue recognition timing variances[cite: 3].

```
In [3]: # Extract clean tables from data dictionary
entities_df = data_dict['entities']
projects_df = data_dict['projects']
timesheets_df = data_dict['timesheets']
invoices_df = data_dict['invoices']
gl_revenue_df = data_dict['gl_revenue']
```

```

# -----
# A. FY2025 Entity Performance (Tasks 3, 4, 5)
# -----
inv_2025 = invoices_df[invoices_df['invoice_date'].str.startswith('2025')].merge(
    projects_df[['project_id', 'entity_code', 'clean_service_line']], on='project_id'
)

ts_2025 = timesheets_df[timesheets_df['week_commencing'].str.startswith('2025')].merge(
    projects_df[['project_id', 'entity_code', 'clean_service_line']], on='project_id'
)

entity_rev = inv_2025.groupby('entity_code')['invoice_amount'].agg(
    invoiced_revenue_2025='sum',
    invoice_count='count'
).reset_index()

entity_del = ts_2025.groupby('entity_code').agg(
    total_hours=('hours', 'sum'),
    perm_hours=('hours', lambda x: x[ts_2025['engagement_type'] == 'Permanent'].sum()),
    contractor_hours=('hours', lambda x: x[ts_2025['engagement_type'] == 'Contract'].sum()),
    known_cost_gbp=('known_cost_gbp', 'sum'),
    imputed_cost_gbp=('imputed_cost_gbp', 'sum')
).reset_index()

entity_summary = entity_rev.merge(entity_del, on='entity_code')
entity_summary['known_margin_gbp'] = entity_summary['invoiced_revenue_2025'] - entity_summary['known_cost_gbp']
entity_summary['known_margin_pct'] = (entity_summary['known_margin_gbp'] / entity_summary['invoiced_revenue_2025'])

entity_summary['imputed_margin_gbp'] = entity_summary['invoiced_revenue_2025'] - entity_summary['imputed_cost_gbp']
entity_summary['imputed_margin_pct'] = (entity_summary['imputed_margin_gbp'] / entity_summary['invoiced_revenue_2025'])

print("=== 1. FY2025 ENTITY PERFORMANCE SUMMARY ===")
display(entity_summary[['entity_code', 'invoiced_revenue_2025', 'total_hours', 'perm_hours', 'contractor_hours', 'known_cost_gbp', 'imputed_cost_gbp', 'known_margin_gbp', 'imputed_margin_gbp', 'known_margin_pct', 'imputed_margin_pct']])

# -----
# B. FY2025 Service Line Profitability (Task 6)
# -----
sline_rev = inv_2025.groupby('clean_service_line')['invoice_amount'].sum().reset_index()
sline_del = ts_2025.groupby('clean_service_line').agg(
    hours=('hours', 'sum'),
    known_cost=('known_cost_gbp', 'sum')
)

```

```

).reset_index()

sline_summary = sline_rev.merge(sline_del, on='clean_service_line')
sline_summary['margin_gbp'] = sline_summary['invoice_amount'] - sline_summary['known_cost']
sline_summary['margin_pct'] = (sline_summary['margin_gbp'] / sline_summary['invoice_amount']) * 100

print("\n=== 2. FY2025 SERVICE LINE PROFITABILITY ===")
display(sline_summary.sort_values(by='invoice_amount', ascending=False))

# -----
# C. YoY Group Trend Analysis (Task 7)
# -----

rev_2024 = invoices_df[invoices_df['invoice_date'].str.startswith('2024')]['invoice_amount'].sum()
cost_2024 = timesheets_df[timesheets_df['week_commencing'].str.startswith('2024')]['known_cost_gbp'].sum()
margin_2024_gbp = rev_2024 - cost_2024
margin_2024_pct = (margin_2024_gbp / rev_2024) * 100

rev_2025 = entity_summary['invoiced_revenue_2025'].sum()
cost_2025 = entity_summary['known_cost_gbp'].sum()
margin_2025_gbp = rev_2025 - cost_2025
margin_2025_pct = (margin_2025_gbp / rev_2025) * 100

yoy_df = pd.DataFrame([
    {
        'Metric': 'Invoiced Revenue (£)',
        'FY2024': rev_2024,
        'FY2025': rev_2025,
        'YoY Growth (£)': rev_2025 - rev_2024,
        'YoY Growth (%)': ((rev_2025 - rev_2024) / rev_2024) * 100
    }, {
        'Metric': 'Direct Labor Cost (£)',
        'FY2024': cost_2024,
        'FY2025': cost_2025,
        'YoY Growth (£)': cost_2025 - cost_2024,
        'YoY Growth (%)': ((cost_2025 - cost_2024) / cost_2024) * 100
    }, {
        'Metric': 'Gross Margin (£)',
        'FY2024': margin_2024_gbp,
        'FY2025': margin_2025_gbp,
        'YoY Growth (£)': margin_2025_gbp - margin_2024_gbp,
        'YoY Growth (%)': ((margin_2025_gbp - margin_2024_gbp) / margin_2024_gbp) * 100
    }, {
        'Metric': 'Gross Margin (%)',

```

```

        'FY2024': margin_2024_pct,
        'FY2025': margin_2025_pct,
        'YoY Growth (£)': margin_2025_pct - margin_2024_pct,
        'YoY Growth (%)': margin_2025_pct - margin_2024_pct
    })

print("\n=== 3. YOY FINANCIAL TREND (FY2024 vs FY2025) ===")
display(yoy_df)

# -----
# D. Revenue Reconciliation to GL (Tasks 8 & 9)
# -----
gl_2025 = gl_revenue_df[gl_revenue_df['period_month'].str.startswith('2025')].groupby('entity_code')['revenue_recognised']

recon_summary = entity_rev[['entity_code', 'invoiced_revenue_2025']].merge(gl_2025, on='entity_code')
recon_summary['variance_gbp'] = recon_summary['invoiced_revenue_2025'] - recon_summary['revenue_recognised']
recon_summary['variance_pct'] = (recon_summary['variance_gbp'] / recon_summary['revenue_recognised']) * 100

print("\n=== 4. REVENUE RECONCILIATION (INVOICES vs GENERAL LEDGER) ===")
display(recon_summary)

```

=== 1. FY2025 ENTITY PERFORMANCE SUMMARY ===

	entity_code	invoiced_revenue_2025	total_hours	perm_hours	contractor_hours	known_cost_gbp	known_margin_pct	imputed_i
0	NB-01	9713447.51	202338.80	186493.10	15845.70	7853992.55	19.14	
1	NB-02	7276258.98	142813.90	133858.10	8955.80	6067810.59	16.61	
2	NB-03	7361091.52	135196.40	118775.60	16420.80	5518223.91	25.04	

=== 2. FY2025 SERVICE LINE PROFITABILITY ===

	clean_service_line	invoice_amount	hours	known_cost	margin_gbp	margin_pct
5	Mechanical & Electrical	6847555.90	141708.60	5521461.86	1326094.04	19.37
4	Maintenance Contracts	6092829.21	108396.90	4440999.80	1651829.41	27.11
0	Advisory	5748544.16	109187.20	4639420.07	1109124.09	19.29
1	Civil Works	2826488.78	60672.30	2343440.45	483048.33	17.09
2	Data & Analytics	1674263.46	36418.50	1529315.55	144947.91	8.66
3	Facilities Support	1161116.50	23965.60	965389.32	195727.18	16.86

=== 3. YOY FINANCIAL TREND (FY2024 vs FY2025) ===

	Metric	FY2024	FY2025	YoY Growth (£)	YoY Growth (%)
0	Invoiced Revenue (£)	21030009.67	24350798.01	3320788.34	15.79
1	Direct Labor Cost (£)	13479372.73	19440027.05	5960654.32	44.22
2	Gross Margin (£)	7550636.94	4910770.96	-2639865.98	-34.96
3	Gross Margin (%)	35.90	20.17	-15.74	-15.74

=== 4. REVENUE RECONCILIATION (INVOICES vs GENERAL LEDGER) ===

	entity_code	invoiced_revenue_2025	revenue recognised	variance_gbp	variance_pct
0	NB-01	9713447.51	7827213.57	1886233.94	24.10
1	NB-02	7276258.98	7221439.40	54819.58	0.76
2	NB-03	7361091.52	7263605.00	97486.52	1.34

3. Financial Analytics & Business Intelligence (Part A Deliverables)

What this section does: Executes core analytical calculations for Part A:

- **FY2025 Entity Performance:** Invoiced Revenue, Total Delivery Hours, Permanent vs. Contractor Hours, Direct Delivery Costs, and Gross Margin % (Known vs. Imputed)[cite: 3].
- **FY2025 Service Line Profitability:** Aggregates financial performance across cleaned service lines[cite: 3].
- **Year-over-Year Trend Analysis (FY2024 vs. FY2025):** Evaluates revenue growth against delivery cost expansion[cite: 3].
- **General Ledger Reconciliation:** Compares operational invoice revenue against `gl_revenue_summary` to quantify revenue recognition timing variances[cite: 3].

```
In [4]: # Extract clean tables from data dictionary
entities_df = data_dict['entities']
projects_df = data_dict['projects']
timesheets_df = data_dict['timesheets']
invoices_df = data_dict['invoices']
gl_revenue_df = data_dict['gl_revenue']

# -----
# A. FY2025 Entity Performance (Tasks 3, 4, 5)
# -----
inv_2025 = invoices_df[invoices_df['invoice_date'].str.startswith('2025')].merge(
    projects_df[['project_id', 'entity_code', 'clean_service_line']], on='project_id'
)

ts_2025 = timesheets_df[timesheets_df['week_commencing'].str.startswith('2025')].merge(
    projects_df[['project_id', 'entity_code', 'clean_service_line']], on='project_id'
)

entity_rev = inv_2025.groupby('entity_code')['invoice_amount'].agg(
    invoiced_revenue_2025='sum',
    invoice_count='count'
).reset_index()

entity_del = ts_2025.groupby('entity_code').agg(
    total_hours=('hours', 'sum'),
    perm_hours=('hours', lambda x: x[ts_2025['engagement_type'] == 'Permanent'].sum()),
    contractor_hours=('hours', lambda x: x[ts_2025['engagement_type'] == 'Contract'].sum()),
    known_cost_gbp=('known_cost_gbp', 'sum'),
    imputed_cost_gbp=('imputed_cost_gbp', 'sum')
).reset_index()

entity_summary = entity_rev.merge(entity_del, on='entity_code')
entity_summary['known_margin_gbp'] = entity_summary['invoiced_revenue_2025'] - entity_summary['known_cost_gbp']
```

```

entity_summary['known_margin_pct'] = (entity_summary['known_margin_gbp'] / entity_summary['invoiced_revenue_2025']) * 100

entity_summary['imputed_margin_gbp'] = entity_summary['invoiced_revenue_2025'] - entity_summary['imputed_cost_gbp']
entity_summary['imputed_margin_pct'] = (entity_summary['imputed_margin_gbp'] / entity_summary['invoiced_revenue_2025']) * 100

print("=== 1. FY2025 ENTITY PERFORMANCE SUMMARY ===")
display(entity_summary[['entity_code', 'invoiced_revenue_2025', 'total_hours', 'perm_hours', 'contractor_hours', 'known_cost_gbp', 'imputed_cost_gbp', 'known_margin_gbp', 'imputed_margin_gbp', 'known_margin_pct', 'imputed_margin_pct']])

# -----
# B. FY2025 Service Line Profitability (Task 6)
# -----
sline_rev = inv_2025.groupby('clean_service_line')['invoice_amount'].sum().reset_index()
sline_del = ts_2025.groupby('clean_service_line').agg(
    hours=('hours', 'sum'),
    known_cost=('known_cost_gbp', 'sum')
).reset_index()

sline_summary = sline_rev.merge(sline_del, on='clean_service_line')
sline_summary['margin_gbp'] = sline_summary['invoice_amount'] - sline_summary['known_cost']
sline_summary['margin_pct'] = (sline_summary['margin_gbp'] / sline_summary['invoice_amount']) * 100

print("\n=== 2. FY2025 SERVICE LINE PROFITABILITY ===")
display(sline_summary.sort_values(by='invoice_amount', ascending=False))

# -----
# C. YoY Group Trend Analysis (Task 7)
# -----
rev_2024 = invoices_df[invoices_df['invoice_date'].str.startswith('2024')]['invoice_amount'].sum()
cost_2024 = timesheets_df[timesheets_df['week_commencing'].str.startswith('2024')]['known_cost_gbp'].sum()
margin_2024_gbp = rev_2024 - cost_2024
margin_2024_pct = (margin_2024_gbp / rev_2024) * 100

rev_2025 = entity_summary['invoiced_revenue_2025'].sum()
cost_2025 = entity_summary['known_cost_gbp'].sum()
margin_2025_gbp = rev_2025 - cost_2025
margin_2025_pct = (margin_2025_gbp / rev_2025) * 100

yoy_df = pd.DataFrame([
    {'Metric': 'Invoiced Revenue (£)',
     'FY2024': rev_2024,
     'FY2025': rev_2025,
     'YoY Growth (£)': rev_2025 - rev_2024,
    },
    {'Metric': 'Known Cost (£)',
     'FY2024': cost_2024,
     'FY2025': cost_2025,
     'YoY Growth (£)': cost_2025 - cost_2024,
    },
    {'Metric': 'Margin (£)',
     'FY2024': margin_2024_gbp,
     'FY2025': margin_2025_gbp,
     'YoY Growth (£)': margin_2025_gbp - margin_2024_gbp,
    },
    {'Metric': 'Margin (%)',
     'FY2024': margin_2024_pct,
     'FY2025': margin_2025_pct,
     'YoY Growth (%)': margin_2025_pct - margin_2024_pct,
    },
])
display(yoy_df)

```

```

        'YoY Growth (%)': ((rev_2025 - rev_2024) / rev_2024) * 100
    }, {
        'Metric': 'Direct Labor Cost (£)',
        'FY2024': cost_2024,
        'FY2025': cost_2025,
        'YoY Growth (£)': cost_2025 - cost_2024,
        'YoY Growth (%)': ((cost_2025 - cost_2024) / cost_2024) * 100
    }, {
        'Metric': 'Gross Margin (£)',
        'FY2024': margin_2024_gbp,
        'FY2025': margin_2025_gbp,
        'YoY Growth (£)': margin_2025_gbp - margin_2024_gbp,
        'YoY Growth (%)': ((margin_2025_gbp - margin_2024_gbp) / margin_2024_gbp) * 100
    }, {
        'Metric': 'Gross Margin (%)',
        'FY2024': margin_2024_pct,
        'FY2025': margin_2025_pct,
        'YoY Growth (£)': margin_2025_pct - margin_2024_pct,
        'YoY Growth (%)': margin_2025_pct - margin_2024_pct
    })
])

print("\n=== 3. YOY FINANCIAL TREND (FY2024 vs FY2025) ===")
display(yoy_df)

# -----
# D. Revenue Reconciliation to GL (Tasks 8 & 9)
# -----
gl_2025 = gl_revenue_df[gl_revenue_df['period_month'].str.startswith('2025')].groupby('entity_code')['revenue_recognised']

recon_summary = entity_rev[['entity_code', 'invoiced_revenue_2025']].merge(gl_2025, on='entity_code')
recon_summary['variance_gbp'] = recon_summary['invoiced_revenue_2025'] - recon_summary['revenue_recognised']
recon_summary['variance_pct'] = (recon_summary['variance_gbp'] / recon_summary['revenue_recognised']) * 100

print("\n=== 4. REVENUE RECONCILIATION (INVOICES vs GENERAL LEDGER) ===")
display(recon_summary)

```

=== 1. FY2025 ENTITY PERFORMANCE SUMMARY ===

	entity_code	invoiced_revenue_2025	total_hours	perm_hours	contractor_hours	known_cost_gbp	known_margin_pct	imputed_i
0	NB-01	9713447.51	202338.80	186493.10	15845.70	7853992.55	19.14	
1	NB-02	7276258.98	142813.90	133858.10	8955.80	6067810.59	16.61	
2	NB-03	7361091.52	135196.40	118775.60	16420.80	5518223.91	25.04	



=== 2. FY2025 SERVICE LINE PROFITABILITY ===

	clean_service_line	invoice_amount	hours	known_cost	margin_gbp	margin_pct
5	Mechanical & Electrical	6847555.90	141708.60	5521461.86	1326094.04	19.37
4	Maintenance Contracts	6092829.21	108396.90	4440999.80	1651829.41	27.11
0	Advisory	5748544.16	109187.20	4639420.07	1109124.09	19.29
1	Civil Works	2826488.78	60672.30	2343440.45	483048.33	17.09
2	Data & Analytics	1674263.46	36418.50	1529315.55	144947.91	8.66
3	Facilities Support	1161116.50	23965.60	965389.32	195727.18	16.86

=== 3. YOY FINANCIAL TREND (FY2024 vs FY2025) ===

	Metric	FY2024	FY2025	YoY Growth (£)	YoY Growth (%)
0	Invoiced Revenue (£)	21030009.67	24350798.01	3320788.34	15.79
1	Direct Labor Cost (£)	13479372.73	19440027.05	5960654.32	44.22
2	Gross Margin (£)	7550636.94	4910770.96	-2639865.98	-34.96
3	Gross Margin (%)	35.90	20.17	-15.74	-15.74

=== 4. REVENUE RECONCILIATION (INVOICES vs GENERAL LEDGER) ===

	entity_code	invoiced_revenue_2025	revenue recognised	variance_gbp	variance_pct
0	NB-01	9713447.51	7827213.57	1886233.94	24.10
1	NB-02	7276258.98	7221439.40	54819.58	0.76
2	NB-03	7361091.52	7263605.00	97486.52	1.34

4. Dimensional Data Warehouse Export (Star Schema for Power BI)

What this section does: Exports the cleaned, transformed DataFrames into a structured relational SQLite database (Northbridge_DW.db) designed as a **Star Schema** for Power BI ingestion[cite: 3].

- **Dimensions:** dim_entities , dim_clients , dim_projects , dim_staff , dim_date
- **Facts:** fact_invoices , fact_invoice_lines , fact_timesheets , fact_gl_revenue

```
In [6]: def export_star_schema(data_dictionary, output_sqlite_file):
        """
        Exports clean star schema tables into a dedicated SQLite database for Power BI.
        """
        dw_conn = sqlite3.connect(output_sqlite_file)

        # 1. Dimensions
        data_dictionary['entities'].to_sql('dim_entities', dw_conn, if_exists='replace', index=False)
        data_dictionary['clients'].to_sql('dim_clients', dw_conn, if_exists='replace', index=False)

        # FIX: Using 'clean_service_line' matching Cell 2 definition
        clean_projects = data_dictionary['projects'][[
            'project_id', 'client_id', 'entity_code', 'project_name',
            'clean_service_line', 'start_date', 'end_date', 'contract_value', 'status'
        ]]
        clean_projects.to_sql('dim_projects', dw_conn, if_exists='replace', index=False)

        data_dictionary['staff'].to_sql('dim_staff', dw_conn, if_exists='replace', index=False)

        # Generate Date Dimension
        date_range = pd.date_range(start='2024-01-01', end='2025-12-31', freq='D')
```

```

dim_date = pd.DataFrame({
    'date_key': date_range.strftime('%Y%m%d').astype(int),
    'full_date': date_range.strftime('%Y-%m-%d'),
    'year': date_range.year,
    'quarter': 'Q' + date_range.quarter.astype(str),
    'month_num': date_range.month,
    'month_name': date_range.strftime('%B'),
    'year_month': date_range.strftime('%Y-%m')
})
dim_date.to_sql('dim_date', dw_conn, if_exists='replace', index=False)

# 2. Facts
data_dictionary['invoices'].to_sql('fact_invoices', dw_conn, if_exists='replace', index=False)
data_dictionary['invoice_lines'].to_sql('fact_invoice_lines', dw_conn, if_exists='replace', index=False)
data_dictionary['timesheets'].to_sql('fact_timesheets', dw_conn, if_exists='replace', index=False)
data_dictionary['gl_revenue'].to_sql('fact_gl_revenue', dw_conn, if_exists='replace', index=False)

print(f"🎉 Star Schema Data Warehouse successfully built at: {output_sqlite_file}")

# Execute Data Warehouse Export
export_star_schema(data_dict, 'Northbridge_DW.db')

```

🎉 Star Schema Data Warehouse successfully built at: Northbridge_DW.db

```

In [7]: import sqlite3
import pandas as pd
import os

# 1. Define folder path & raw database path
folder_path = r'C:\Users\Remote-07\Downloads\RSM-Power-House-DBA-Candidate-Pack-v1.2'
db_path = os.path.join(folder_path, 'Northbridge_Assessment.db')

# 2. Connect to raw SQLite database
conn = sqlite3.connect(db_path)

# 3. Load raw tables into DataFrames
entities = pd.read_sql_query("SELECT * FROM entities", conn)
clients = pd.read_sql_query("SELECT * FROM clients", conn)
projects = pd.read_sql_query("SELECT * FROM projects", conn)
staff = pd.read_sql_query("SELECT * FROM staff", conn)
timesheets = pd.read_sql_query("SELECT * FROM timesheets", conn)
invoices = pd.read_sql_query("SELECT * FROM invoices", conn)

```

```
invoice_lines = pd.read_sql_query("SELECT * FROM invoice_lines", conn)
gl_revenue = pd.read_sql_query("SELECT * FROM gl_revenue_summary", conn)

# 4. Clean & Transform Data
# A. Trim whitespace
entities['entity_code'] = entities['entity_code'].str.strip()
projects['entity_code'] = projects['entity_code'].str.strip()
staff['entity_code'] = staff['entity_code'].str.strip()
gl_revenue['entity_code'] = gl_revenue['entity_code'].str.strip()

# B. Unify service lines
service_line_map = {
    'M&E': 'Mechanical & Electrical',
    'Consulting': 'Advisory'
}
projects['clean_service_line'] = projects['service_line'].replace(service_line_map)

# C. Impute contractor cost rates
ts_staff = timesheets.merge(staff[['staff_id', 'grade', 'engagement_type']], on='staff_id', how='left')

perm_grade_rates = (
    ts_staff[ts_staff['engagement_type'] == 'Permanent']
    .groupby('grade')['cost_rate']
    .mean()
    .to_dict()
)

ts_staff['imputed_cost_rate'] = ts_staff.apply(
    lambda row: row['cost_rate'] if pd.notnull(row['cost_rate'])
    else perm_grade_rates.get(row['grade'], 0.0),
    axis=1
)

ts_staff['known_cost_gbp'] = ts_staff['hours'] * ts_staff['cost_rate'].fillna(0.0)
ts_staff['imputed_cost_gbp'] = ts_staff['hours'] * ts_staff['imputed_cost_rate']

# D. Build Date Dimension
date_range = pd.date_range(start='2024-01-01', end='2025-12-31', freq='D')
dim_date = pd.DataFrame({
    'date_key': date_range.strftime('%Y%m%d').astype(int),
    'full_date': date_range.strftime('%Y-%m-%d'),
    'year': date_range.year,
```



```

        'quarter': 'Q' + date_range.quarter.astype(str),
        'month_num': date_range.month,
        'month_name': date_range.strftime('%B'),
        'year_month': date_range.strftime('%Y-%m')
    })

# 5. Export clean CSV files into folder
csv_exports = {
    'dim_entities.csv': entities,
    'dim_clients.csv': clients,
    'dim_projects.csv': projects[['project_id', 'client_id', 'entity_code', 'project_name', 'clean_service_line', 'st
    'dim_staff.csv': staff,
    'dim_date.csv': dim_date,
    'fact_invoices.csv': invoices,
    'fact_invoice_lines.csv': invoice_lines,
    'fact_timesheets.csv': ts_staff,
    'fact_gl_revenue.csv': gl_revenue
}

print("=== EXPORTING CLEAN CSV FILES FOR POWER BI ===")
for filename, df in csv_exports.items():
    file_path = os.path.join(folder_path, filename)
    df.to_csv(file_path, index=False)
    print(f"✅ Exported: {filename} ({len(df):,d} rows)")

conn.close()
print("\n🎉 ALL CSV FILES ARE CREATED AND READY FOR POWER BI IMPORT!")

```

=== EXPORTING CLEAN CSV FILES FOR POWER BI ===

- ✅ Exported: dim_entities.csv (3 rows)
- ✅ Exported: dim_clients.csv (180 rows)
- ✅ Exported: dim_projects.csv (640 rows)
- ✅ Exported: dim_staff.csv (210 rows)
- ✅ Exported: dim_date.csv (731 rows)
- ✅ Exported: fact_invoices.csv (2,339 rows)
- ✅ Exported: fact_invoice_lines.csv (7,010 rows)
- ✅ Exported: fact_timesheets.csv (35,634 rows)
- ✅ Exported: fact_gl_revenue.csv (72 rows)

🎉 ALL CSV FILES ARE CREATED AND READY FOR POWER BI IMPORT!

In []:

